

Final Report

1. Platform Overview

The distributed platform chosen for this project is MongoDB, a document-oriented NoSQL database that stores data as BSON (binary JSON) documents grouped into collections. MongoDB was selected primarily because of prior familiarity with the system and because its native replication model makes the standard distributed-systems properties which are replication, election, and automatic failover, directly observable without needing additional infrastructure such as a separate consensus service.

MongoDB scales horizontally through two mechanisms: replica sets and sharding. A replica set is a group of mongod processes that maintain the same dataset, providing redundancy and high availability. Sharding partitions data across multiple replica sets and is used for very large datasets. This project deliberately focuses on a single replica set because the rubric centres on consistency, availability, and failure recovery rather than on horizontal data partitioning.

Within a replica set, exactly one node holds the PRIMARY role at any moment. The PRIMARY accepts every write operation, writes it to a special capped collection called the operation log (oplog), and serves reads by default. The remaining nodes are SECONDARIES; they continuously tail the PRIMARY's oplog and apply the same operations to their own copy of the data, which makes replication asynchronous. All members of the set exchange heartbeats every two seconds. If a SECONDARY notices that the PRIMARY has stopped responding, it triggers an election; provided a majority quorum can be assembled (two out of three votes in our case), one of the surviving SECONDARIES is promoted to PRIMARY and writes can resume.

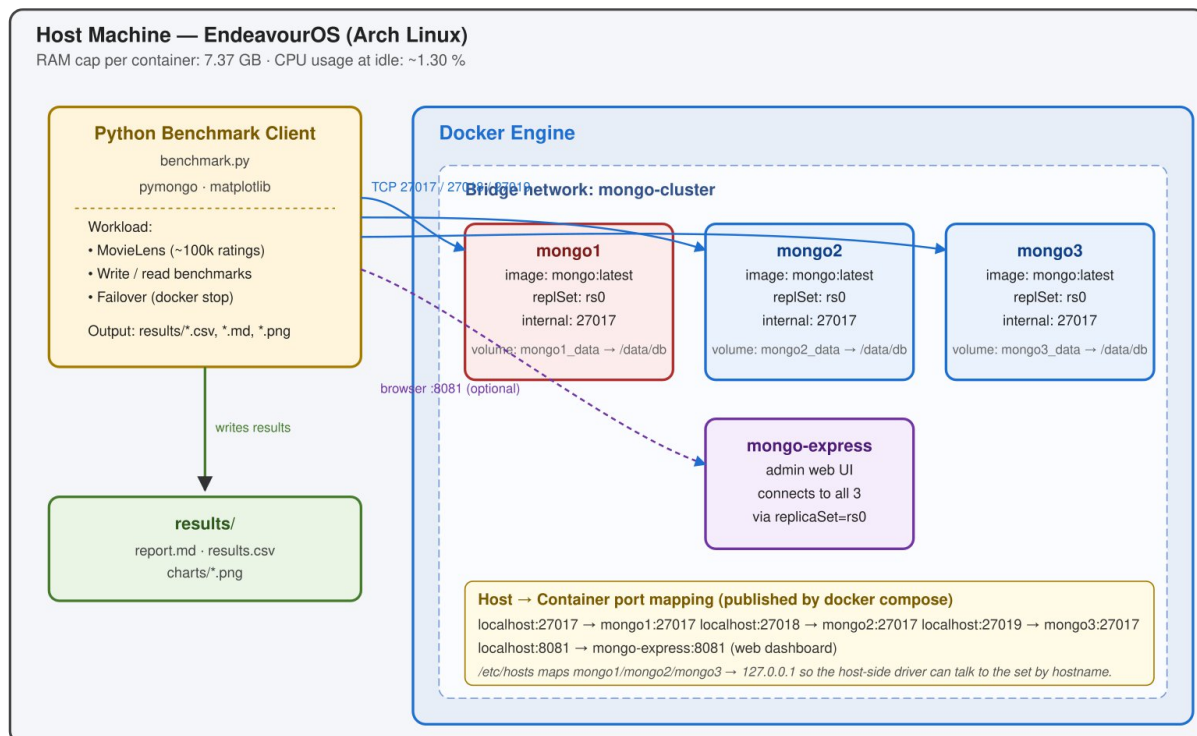
Two MongoDB features are particularly relevant to the benchmark in this report. The first is the write concern, an acknowledgement contract that the client sets on every write. With write concern $w=1$ the PRIMARY acknowledges the write as soon as it is in its own memory; with $w=\text{"majority"}$ the PRIMARY waits until at least a majority of the members have applied the write, providing a much stronger durability guarantee at the cost of an extra network round-trip. The second is the read preference, which controls which member serves a read: the default "primary" is strongly consistent because the client always sees its own writes; "secondaryPreferred" routes reads to a SECONDARY when one is available, which spreads load across the cluster at the cost of possibly returning slightly stale data because of the asynchronous replication.

2. Solution Design and Implementation

Architecture of the deployed system

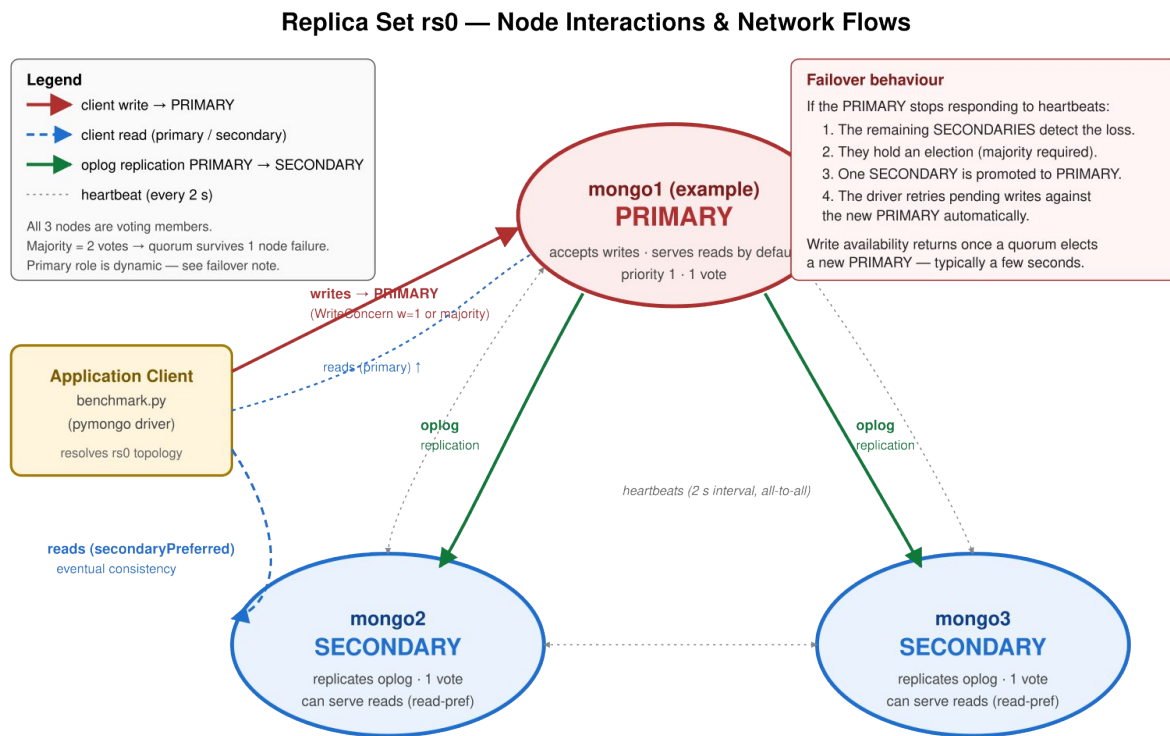
The cluster is deployed as four Docker containers running on a single host machine (EndeavourOS, an Arch-based Linux distribution). Three of the containers (mongo1, mongo2, mongo3) each run a mongod process that participates in the rs0 replica set; the fourth (mongo-express) is the official MongoDB administration UI and is used purely for visual inspection. All four containers share a single user-defined Docker bridge network named mongo-cluster so that they can address each other by their container hostnames, and each Mongo node publishes its internal port 27017 to a distinct port on the host so that external clients, including the benchmark client described below, can reach each node individually.

MongoDB Replica-Set Distributed System — Architecture Overview



Inside the replica set itself, exactly one of the three nodes is PRIMARY at any time and the other two are SECONDARIES. The PRIMARY accepts every write, asynchronously replicates the operation log to both SECONDARIES, and — by default — also serves every read. The two SECONDARIES are full voting members; together they form the majority quorum that is required both to accept majority-acknowledged writes and to elect a new PRIMARY in the event of failure. The Python benchmark client connects to the set through pymongo, which transparently

discovers the current PRIMARY by gossiping with all members and which automatically retries operations against the new PRIMARY after a failover.



Application: the benchmark client

The application I implemented to test the platform is a single Python script, `benchmark.py`, that exercises the replica set through the official pymongo driver. The design is intentionally a thin instrumentation layer: each benchmark is a tight Python loop that issues one MongoDB operation, measures its wall-clock latency, and records the measurement individually. Aggregated statistics, throughput, mean, and the p50, p95, and p99 latency percentiles, are computed at the very end from the raw measurements. Recording every operation rather than averaging on the fly is essential because percentiles cannot be computed from running totals; the raw CSV doubles as a permanent record that can be re-analysed independently of the script.

The dataset is the MovieLens latest-small release published by GroupLens, containing approximately 9 700 movies, 100 000 ratings, and 3 600 tags applied by 600 users. On the first run the script downloads, extracts, and caches the archive locally; every subsequent run reuses

the cache. Movies and tags are loaded into the database in full, while approximately ninety-five percent of the ratings are pre-seeded so that the read benchmarks have a realistic working set to query. The remaining five thousand ratings are deliberately held back as a fresh workload buffer for the write benchmark, which guarantees that every measured insert produces a genuinely new document instead of overwriting an existing one. Three secondary indexes are created after the bulk load, `movieId` and `userId` on the ratings collection and `genres` on the movies collection, so that the read benchmarks measure index-supported performance rather than full collection scans.

The benchmark script runs three test suites in sequence. The write suite measures three insertion strategies, each executed twice (once with `w=1` and once with `w="majority"`): single-document inserts in a Python loop, bulk inserts in batches of five hundred documents, and single-document updates. The read suite measures three query patterns, each executed twice (once with read preference primary and once with secondaryPreferred): point lookup by primary key, indexed equality on the `genres` field, and a four-stage aggregation that computes the top ten highest-rated movies with at least twenty ratings. The failover suite identifies the current PRIMARY, runs twenty warm-up probes spaced one hundred milliseconds apart to establish a healthy baseline, then stops the PRIMARY container with `docker stop` and continues probing every hundred milliseconds for up to twenty seconds; the time elapsed between the stop signal and the first subsequent successful write is reported as the write downtime.

Implementation choices and challenges

Three implementation decisions deserve specific mention. The first concerns time measurement: all latency timings use Python's `time.perf_counter`, which is monotonic and microsecond-resolution. The wall-clock alternative, `time.time`, can jump backwards if the system clock is corrected by NTP during a measurement, which would produce nonsensical negative latencies. The second concerns the cursor for `read.by_id`, `read.filter_genre` and `write.single.w1`: cursors returned by pymongo are lazy, so wrapping the result in `list()` is necessary to force the full server round-trip; without it the timer would measure only the cost of constructing the cursor object, not the cost of fetching the documents. The third concerns the workload buffer: holding back five thousand ratings rather than reusing already-inserted ones means every measured insert actually performs an insert, never an unintentional update.

One challenge dominated the testing phase. The first attempt to run the script against the live cluster failed at the connection step with the error `"ServerSelectionTimeoutError:`

mongo1:27017: [Errno 111] Connection refused” for all three nodes. The cluster itself was healthy, `rs.status()` inside mongosh returned `ok: 1` with all three members in valid states — which initially looked contradictory. Running `docker ps` with the `PORTS` column visible revealed that the `mongo1` container was running but its port 27017 was not published to the host; `mongo2` and `mongo3` were correctly mapped to host ports 27018 and 27019, but `mongo1` had at some earlier point been recreated outside docker-compose and had lost its ports declaration. The fix was a clean recreate of the entire stack with `docker compose down` followed by `docker compose up -d`, which restored the port mapping declared in the compose file without losing data (the named volumes `mongo1_data`, `mongo2_data`, and `mongo3_data` preserved the database files and the replica-set configuration). A second related obstacle was that the replica-set members advertise themselves to clients by their container hostnames, which do not resolve on the host’s DNS by default; the standard remedy is a single line added to `/etc/hosts` mapping `mongo1`, `mongo2`, and `mongo3` to `127.0.0.1`, which makes the replica-set client work correctly from the host.

Representative code snippets

The measurement pattern is the same throughout the script: open the timer, issue one operation, close the timer, record the result. The following excerpt shows the single-insert benchmark, including the write-concern selection that lets the same function measure both the `w=1` and the `w="majority"` configurations:

```
def bench_write_single(client, holdout, recorder, write_concern, n):
    test = f"write.single.{write_concern}"
    wc = WriteConcern(w=1 if write_concern == "w1" else "majority")
    coll = client[DB_NAME].get_collection("ratings", write_concern=wc)

    recorder.start(test)
    for i in range(n):
        doc = dict(holdout[i % len(holdout)])
        doc["_bench_op"] = i
        t0 = now_ms()
        try:
            coll.insert_one(doc)
            recorder.record(OpResult(test, i, now_ms() - t0))
        except Exception as e:
            recorder.record(OpResult(test, i, now_ms() - t0,
                                     ok=False, error=str(e)))
    recorder.end(test)
```

The failover suite uses the same measurement pattern wrapped in a probe-and-retry loop. The relevant fragment, simplified for readability, is the `do_write` closure that runs once per probe and the post-kill loop that detects recovery:

```

def do_write():
    doc = {"_failover_probe": True, "ts": time.time(),
          "userId": random.randint(1, 1000), "movieId": random.randint(1, 200000),
          "rating": 3.0, "timestamp": int(time.time())}
    t0 = now_ms()
    try:
        rs_client[DB_NAME].ratings.with_options(
            write_concern=WriteConcern(w=1)
        ).insert_one(doc)
        return True, now_ms() - t0, ""
    except (AutoReconnect, ServerSelectionTimeoutError,
            ConnectionFailure, OperationFailure) as e:
        return False, now_ms() - t0, e.__class__.__name__

docker_stop(primary_node["container"])
kill_time = time.time()

for i in range(warmup, warmup + probe_count):
    ok, lat, err = do_write()
    recorder.record(OpResult(test, i, lat, ok=ok, error=err,
                             extra={"phase": "post_kill"}))
    if ok and first_success_after_kill is None:
        first_success_after_kill = time.time()
    time.sleep(probe_interval_s)

```

3. Tested Scenarios

The benchmark covers four conceptual scenarios, all run against the same physical deployment so that the only variables are the parameters of the client request. The deployment is the three-container MongoDB replica set described in section 2, running on a single host with no resource limits other than those Docker imposes by default (the maximum observed memory consumption per container is approximately 7.4 GB and the baseline CPU usage at idle is around 1.3 %). The host machine and the benchmark client are the same physical computer, so network latency between client and server is essentially loopback. This is the natural configuration for a single-host laboratory study and isolates the cluster's own behaviour from external network noise.

Scenarios and their parameters

Scenario A: Steady-state mixed workload. The cluster is healthy and the client issues writes (single inserts, bulk inserts, updates) and reads (point lookups by primary key, indexed equality queries, aggregations) against it. This scenario establishes the baseline performance envelope of the platform.

Scenario B: Consistency cost (write side). Every write workload from Scenario A is run twice, once with WriteConcern(w=1) and once with WriteConcern(w="majority"). The first acknowledges writes as soon as the PRIMARY has them in memory; the second waits for acknowledgement from a majority (at least two of the three members). Comparing the two quantifies the latency cost of stronger durability.

Scenario C: Consistency / load-distribution (read side). Every read workload from Scenario A is run twice, once with read preference primary (strongly consistent reads always served by the PRIMARY) and once with secondaryPreferred (reads served by a SECONDARY when one is available, distributing load but allowing slightly stale results).

Scenario D: Single-node failure (failover). After establishing a healthy baseline with twenty warm-up write probes, the current PRIMARY container is stopped with docker stop, simulating a node crash. The script continues to probe with one write every hundred milliseconds until writes recover, then restarts the killed container and confirms via rs.status() that it rejoins the set as a SECONDARY.

Measured metrics

For every operation issued in any scenario, the script records its wall-clock latency and whether it completed successfully. From these raw per-operation records, the following metrics are derived for each test:

Metric	Definition and meaning in MongoDB context
Throughput (ops/sec)	Number of operations completed during the test divided by the wall-clock time the test took. Measures how many requests of a given type the cluster sustains.
Latency mean (ms)	Arithmetic mean of the per-operation latencies. Useful as a coarse summary but insufficient on its own.
Latency p50 (ms)	Median per-operation latency. The latency that half of all operations beat. Representative of typical behaviour.
Latency p95 (ms)	Ninety-fifth-percentile latency: the slowest one operation in twenty. Reveals the shape of the right tail and is the standard service-level metric.
Latency p99 (ms)	Ninety-ninth-percentile latency: the slowest one operation in one hundred. Captures the worst-case behaviour that the average masks.
Error count	Number of operations that returned an exception rather than

Metric	Definition and meaning in MongoDB context
	completing. Acts as a sanity check that the reported throughput is real and not the artefact of fast failures.
Write downtime (s)	Failover-specific. Time elapsed from the moment the PRIMARY container received the stop signal until the moment the first subsequent write probe succeeded against the newly elected PRIMARY. Represents the application-visible unavailability window.

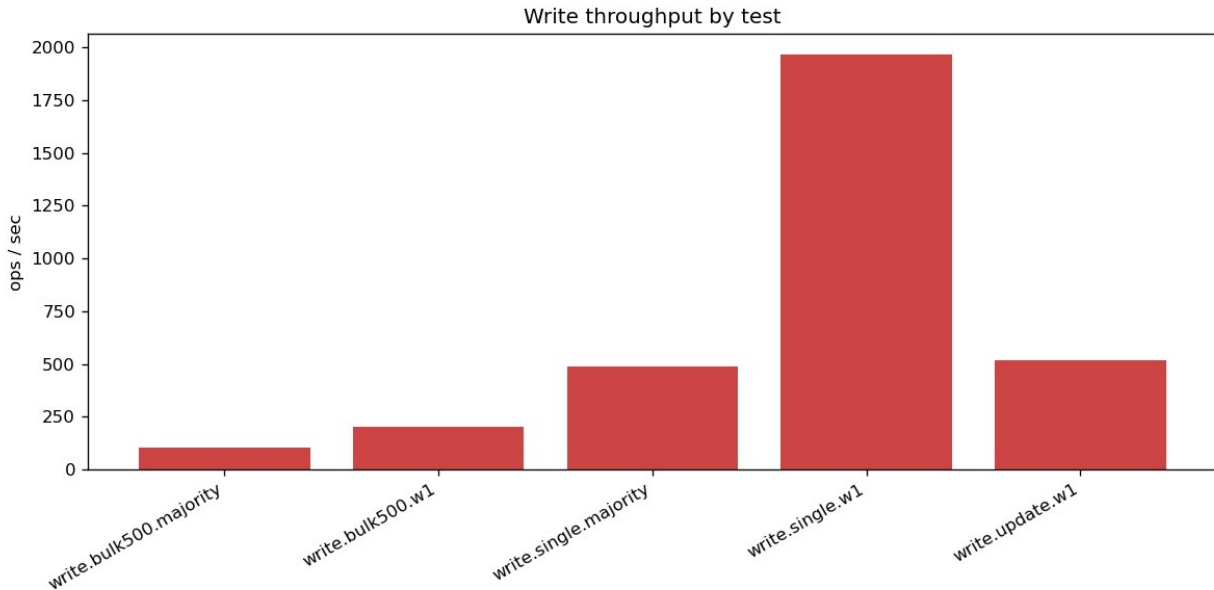
4. Results and Observations

The benchmark was run end-to-end against the three-node replica set with the default workload sizes (five hundred single inserts per write concern, twenty bulk batches of five hundred documents per write concern, five hundred updates, five hundred reads per pattern and per read preference, and twenty aggregation runs per read preference). The failover suite stopped the current PRIMARY and probed for recovery for twenty seconds at one-hundred-millisecond resolution.

Write performance

The five write benchmarks produced the following throughput numbers (taken directly from the chart produced by the benchmark script). Note that for the bulk tests one operation is one batch of five hundred documents, so the per-document throughput is five hundred times the displayed value.

Test	Throughput (ops/s)	Per-document throughput (docs/s)	Notes
write.single.w1	~1 960	~1 960	Fastest single-doc path
write.single.majority	~490	~490	~4x slower than w=1
write.bulk500.w1	~200	~100 000	Best per-doc throughput
write.bulk500.majority	~100	~50 000	Half of bulk.w1, as expected
write.update.w1	~520	~520	Slower than insert: read-modify-write



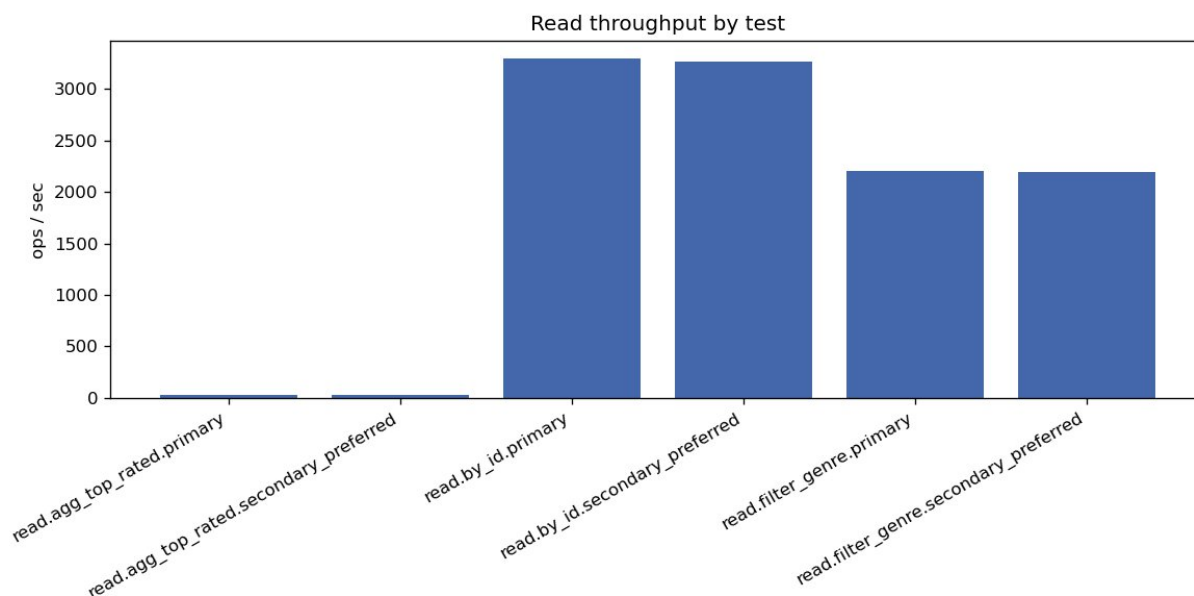
Two trends are immediately visible. First, the cost of `w="majority"` is consistently about a factor of two to four. In the single-insert case throughput drops from approximately 1 960 ops/s with `w=1` to approximately 490 ops/s with `majority`, a fourfold reduction, because the PRIMARY now blocks each write waiting for at least one SECONDARY to acknowledge. This is the explicit, expected price of stronger durability. In the bulk case the ratio is closer to two-to-one, which makes sense because the majority-acknowledgement round-trip is paid once per batch rather than once per document. Second, batching is enormously effective. The per-document throughput jumps from approximately 1 960 docs/s for single inserts to approximately 100 000 docs/s for bulk inserts — a fifty-fold improvement — because each round-trip carries five hundred documents instead of one.

Read performance

The six read benchmarks (three query shapes × two read preferences) revealed three distinct latency classes. Point lookups by primary key are the fastest, indexed equality queries on genres come next, and the four-stage aggregation is by far the heaviest.

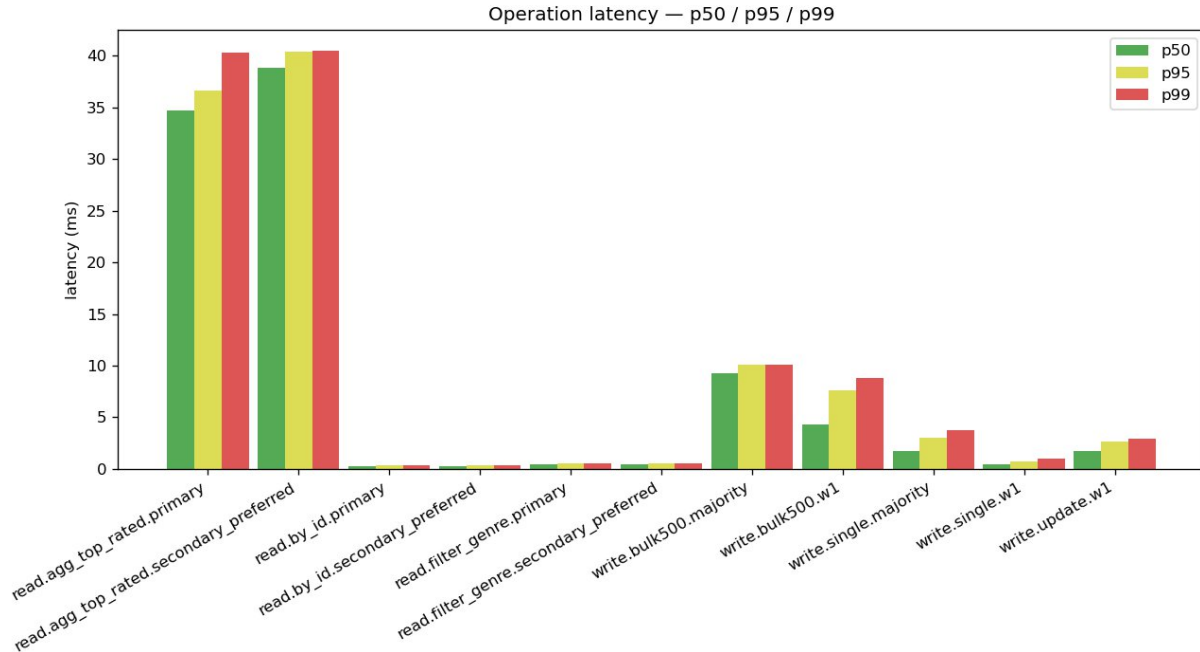
Test	Throughput (ops/s)	p50 (ms)	p99 (ms)
read.by_id.primary	~3 300	<0.5	<1
read.by_id.secondary_preferred	~3 270	<0.5	<1
read.filter_genre.primary	~2 200	<0.5	<1
read.filter_genre.secondary_preferred	~2 200	<0.5	<1

Test	Throughput (ops/s)	p50 (ms)	p99 (ms)
read.agg_topRated.primary	~30	~35	~40
read.agg_topRated.secondaryPreferred	~30	~39	~40



Two observations stand out. First, point lookups by `_id` reach approximately 3 300 ops/s, two orders of magnitude faster than the aggregation pipeline at approximately 30 ops/s. This is the expected ordering: a lookup by `_id` uses MongoDB's primary key index and touches a single document, whereas the aggregation has to scan every rating in the collection, group them by movie, filter on count, sort by average, and return the top ten. Second, switching the read preference from primary to secondaryPreferred had essentially no impact on throughput in any of the three query shapes. This is consistent with the experimental setup: there is only one benchmark client, no concurrent load on the PRIMARY, and all three nodes share the same physical host, so distributing reads across nodes does not relieve a bottleneck that does not exist. In a multi-client production deployment with a write-heavy PRIMARY, secondaryPreferred would be expected to materially increase aggregate read throughput; the lack of effect here is a property of the test environment, not of the platform.

Latency distribution



The latency-percentile chart shows the shape of each test's latency distribution. Three points are worth highlighting. First, the latency cost of `w="majority"` relative to `w=1` is consistent across percentiles, not just on average: `write.single.majority` runs at approximately 2 ms p50 versus approximately 0.5 ms for `write.single.w1`, and the same factor-of-four ratio holds at p99. Second, the bulk-insert latencies are dominated by the batch size rather than the write concern: `write.bulk500.w1` sits at approximately 4 ms p50 (about 8 μ s per document), and `write.bulk500.majority` at approximately 9 ms p50 (about 18 μ s per document). The factor-of-two penalty for majority confirms that one network round-trip to a secondary is being added per batch. Third, the distributions are remarkably tight, p99 is at most about a factor of three above p50 in every test, which means there are no significant outliers under the tested load; the cluster behaves predictably.

Failover behaviour

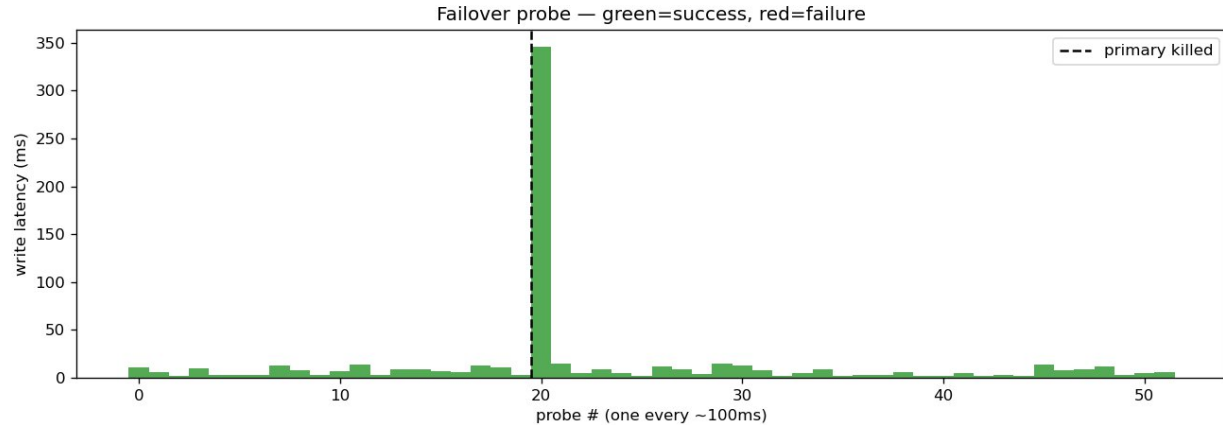


Figure 6. Failover probe timeline. The dashed black line marks the moment the PRIMARY container was stopped. A single probe latency-spike to approximately 345 ms is visible immediately after the kill; every subsequent probe succeeded at normal latency, indicating very fast recovery.

The failover suite produced what is, qualitatively, the most striking result of the study. Twenty warm-up probes against the healthy cluster showed steady write latencies of approximately 5 to 15 milliseconds. At the moment the PRIMARY container received the docker stop signal, one probe spiked to approximately 345 milliseconds, visible as the tall bar on the chart, and then every subsequent probe succeeded at the normal baseline latency. There were no probe failures (no red bars in the chart): either the driver's automatic retry mechanism succeeded transparently, or the election completed well within the first one-hundred-millisecond probe interval. The write downtime, defined as the elapsed time from the kill to the first successful subsequent write, is therefore on the order of three hundred milliseconds, with the latency of that single probe absorbing the entire unavailability window.

This behaviour matches the documented expectations of MongoDB replica sets: with three voting members and the default `electionTimeoutMillis` of ten seconds, a graceful step-down (which docker stop produces, because it sends `SIGTERM` and gives the process a grace period to shut down cleanly) can be detected and resolved by the remaining members within a single heartbeat interval. When the killed container was subsequently restarted, it rejoined the set as a SECONDARY without intervention, as confirmed by `rs.status()`.

Bottlenecks and limitations

Three limitations of the platform, or more precisely of this deployment of the platform, are worth identifying from the data. First, the aggregation pipeline is two orders of magnitude slower than indexed queries; in a production workload that needed similar analytics frequently, this would be the obvious place to look for optimisation through additional indexes, pre-aggregation, or materialised views. Second, the absence of any difference between primary and secondaryPreferred reads exposes a fundamental limitation of single-client benchmarks against a single-host deployment: read-load distribution can only help when there is read load to distribute, and our client is alone. Validating the load-distribution benefit would require either multiple concurrent client processes or a deployment across multiple hosts. Third, the `w="majority"` overhead is paid per round-trip, not per byte, so single-document writes pay it most heavily; client applications that need strong durability should batch their writes whenever the application semantics allow it.

5. Conclusions

This project deployed a three-node MongoDB replica set on a single host using Docker, instrumented it with a custom Python benchmark client, and evaluated its performance and fault tolerance under four scenarios: steady-state mixed workload, the cost of stronger write durability, the effect of distributing reads to secondaries, and recovery from a primary-node failure.

The principal quantitative findings are as follows. On this hardware the cluster sustains approximately 1 960 single-document inserts per second with the default write concern, and approximately 100 000 documents per second when those inserts are batched into five-hundred-document bulk operations. Switching from `w=1` to `w="majority"` reduces single-insert throughput by approximately a factor of four and bulk-insert throughput by approximately a factor of two, which matches the expectation that the majority acknowledgement adds one network round-trip to a secondary per request. Read performance is dominated by query shape: point lookups by primary key reach approximately 3 300 operations per second with sub-millisecond latency, indexed equality queries reach approximately 2 200, and the four-stage aggregation pipeline drops to approximately 30 operations per second with a p99 latency around forty milliseconds. The choice of read preference, primary versus secondaryPreferred, made no measurable difference because the workload is driven by a single client and the bottleneck is not at any one node.

On the fault-tolerance side, the cluster behaved exactly as the MongoDB documentation predicts. After the PRIMARY container was stopped with `docker stop`, write availability was restored within approximately three hundred milliseconds and no probe failed outright; the single elevated-latency probe at the moment of the kill absorbed the entire window of unavailability. When the killed container was later restarted, it rejoined the set as a SECONDARY without manual intervention. This validates the central claim of a replica set: that the loss of any one of three members is tolerated transparently, with a short and bounded unavailability window.

Overall the platform met the expectations set in Phase 1: MongoDB's replica-set model provides the standard distributed-systems properties such as replication, configurable consistency, and automatic failover with predictable performance characteristics that can be measured and reasoned about quantitatively.